# R1.01

CM-01

Courte introduction au langage C

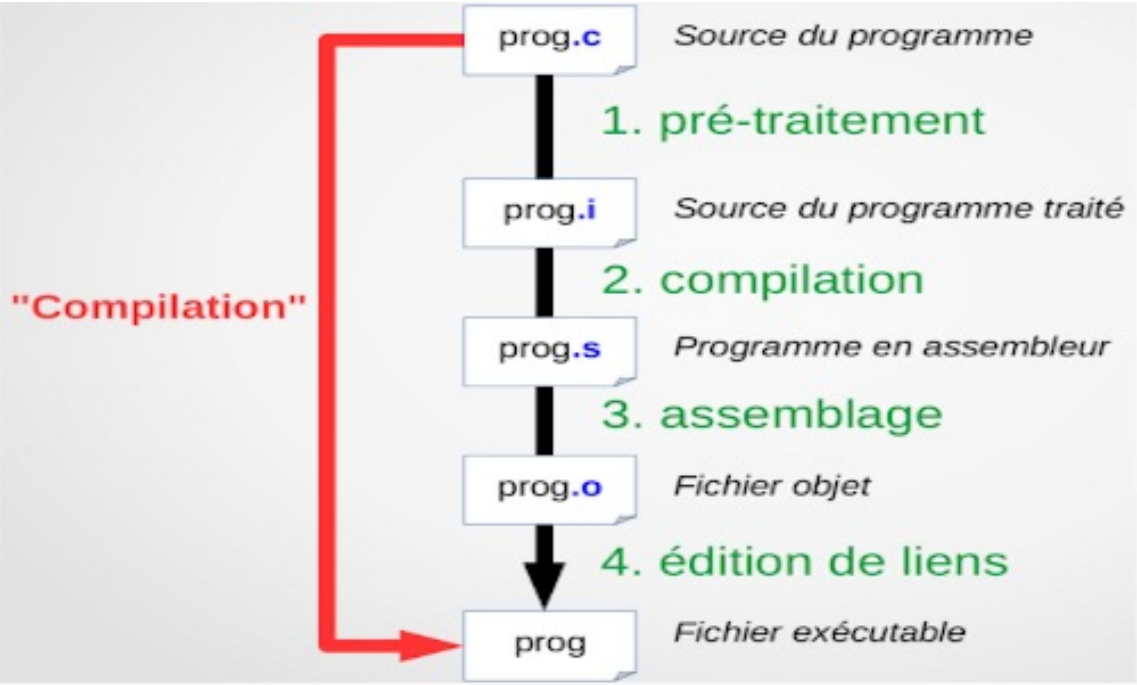
# Langage C

Le langage C est un langage de programmation impératif, généraliste, non objet et de "bas niveau", inventé au début des années 1970.

Nous abordons ce langage en premier pour poser les bases de la programmation et comme un prérequis pour d'autres ressources.

# Notion de compilation

Compilation : Code source lisible par un humain vers un fichier binaire exécutable



# Code source (.c)

```c
//hello.c


//Inclusion de
//En-tête Standard d'Entrée/Sortie
#include <stdio.h>
//Définition d'une macro
#define USER "jub"


//La fonction main
int main(int argc, char *argv[]){
//l'instruction d'appel à la fonction  printf de la bibliothéque stdio
printf("Bonjour %s\n", USER);
//fin du main et renvoie du code retour 0
return 0;
}
```

# Compilation : préprocesseur

Le préprocesseur C ou cpp assure une phase préliminaire de la traduction (compilation) des programmes informatiques écrits dans les langages de programmation C et C++. Comme préprocesseur, il permet principalement l'inclusion d'un segment de code source disponible dans un autre fichier (fichiers d'en-tête ou header), la substitution de chaînes de caractères (macro définition), ainsi que la compilation conditionnelle.

Directives : #include (recopie de fichier), #define (définition de macro et expansion), …

Commande : gcc -E hello.c -o hello.i

# Code après préprocesseur (.i)

```
typedef unsigned char __uint8_t;
typedef short __int16_t;
...
int printf(const char * restrict, ...)
...
int main(int argc, char *argv[]){
printf("Bonjour %s\n", "jub");
return 0;
}
```

# Compilation : compilation

- La compilation proprement dite traduit le fichier généré par le préprocesseur en assembleur, c'est-à-dire en
- une suite d'instructions du microprocesseur qui utilisent des mnémoniques rendant la lecture possible

Commande : gcc -S hello.i

# Code après compilation (.s)

```
.section __TEXT,__text,regular,pure_instructions
.build_version macos, 15, 0 sdk_version 15, 5
.globl _main ; -- Begin function main
.p2align 2 _main: ; @main
.cfi_startproc
; %bb.0:
sub sp, sp, #48
stp x29, x30, [sp, #32] ; 16-byte Folded Spill
add x29, sp, #32
.cfi_def_cfa w29, 16
.cfi_offset w30, -8
.cfi_offset w29, -16
mov w8, #0 ; =0x0
...
ret
.cfi_endproc
; -- End function
…
```

# Compilation : Assemblage

Transforme le code assembleur en un fichier binaire

Commande : gcc -c hello.s

Rem : od -x hello.o pour visualiser

# Code après assemblage (.o)

```
0000000 facf feed 000c 0100 0000 0000 0001 0000
0000020 0004 0000 01b8 0000 2000 0000 0000 0000
0000040 0019 0000 0138 0000 0000 0000 0000 0000
0000060 0000 0000 0000 0000 0000 0000 0000 0000
0000100 0080 0000 0000 0000 01d8 0000 0000 0000
0000120 0080 0000 0000 0000 0007 0000 0007 0000
0000140 0003 0000 0000 0000 5f5f 6574 7478 0000
0000160 0000 0000 0000 0000 5f5f 4554 5458 0000
0000200 0000 0000 0000 0000 0000 0000 0000 0000
0000220 004c 0000 0000 0000 01d8 0000 0002 0000
...
```

# Compilation : Édition de liens

Une fois chaque code source assemblé, il faut donc lier entre eux les différents fichiers objets. L'édition de liens produit alors un fichier dit exécutable.

gcc : gcc -o hello hello.o

# Compilation

- Si un seul fichier sans dépendance :

  gcc –Wall –std=c23 fichier.c –o nom_executable

- Si plusieurs fichiers avec dépendances (include)

  test.c dépend de lib.c

  1. On compile lib.c gcc –wall –std=c23 –c lib.c //produit lib.o
  2. On compile lib.c gcc –wall –std=c23 –c test.c //produit test.o
  3. Edition de lien gcc –wall –std=c23  lib.o test.o –o nom_exe

# Compilation : makefile

But maintenir les dépendances

```
cible: dépendances
  commande

NAME=exercice1.4
CC=gcc
CFLAGS=-Wall -std=c23
$(NAME): $(NAME).lib.o $(NAME).test.o
  $(CC) $(CFLAGS) $(NAME).lib.o $(NAME).test.o -o $(NAME)
$(NAME).lib.o: $(NAME).lib.c
  $(CC) $(CFLAGS) -c $(NAME).lib.c
$(NAME).test.o: $(NAME).test.c
  $(CC) $(CFLAGS) -c $(NAME).test.c
```

Rem : pas de tabulation

Commande : make -f le_nom_du_makefile

# Le langage : **Les commentaires**

// Ligne

/*
Bloc
*/

# Le langage : **Les identificateurs**

Les identificateurs permettent de nommer une variable, une fonction, un type défini par typedef, struct, union ou enum, une étiquette. Ils sont composés de lettres, de chiffres et de _.

Ils ne peuvent commencer par un chiffre, ni être des mots clefs du langage.

# Le langage : mots clef

- Les spécificateurs de stockage
  - auto, register, static, extern, typedef
- **Les spécificateurs de type**
  - char, double, enum, float, int, long, short, signed, struct, union, unsigned, void
- Les qualificateurs de type
  - const, volatile
- **Les instructions de contrôle**
  - break, case, continue, default, do, else, for, goto, if, switch, while
- Divers
  - return, sizeof

# Le langage : **Les types prédéfinis**

Un type est un ensemble de valeurs et des opérations sur ces dernières

Caractère (un entier) :

```
char c1 = 'A’;
unsigned char c2 = 'a';
```

Entier :

```
short int a;
//un entier de taille (longeur) comprise entre char et int
int b;
//un entier
long int a;
//un entier de taille supérieure à celle d'un int
unsigned short int;
//un entier court non signé
```

# Le langage : **Les types prédéfinis**

Les flottants :

```
float a; //simple précision
double b; //double précision
long double c; //quadruple précision
```

# Le langage : **Les constantes**

Entières :

```
int a = 10; //décimale
int b = 011; //octale (base 8) commence par 0
int c = 0xA; //hexadécimal (base 16) commence
 par 0x
//seulement sur certains compilateurs
//int c=0b1010; en binaire (base 2) commence par
 0b
```

# Le langage : **Les constantes**

flottant

```
double a = -10.10E-4;
float b = -10.10E-4F; //Le nombre est suffixé par F
long c = -10.10E-4L; //Le nombre est suffixé par F
```

# Le langage : **Les constantes**

Caractère :

- Entre simple quote ‘,

‘,?,\ doivent être échapés : \', \?, \\

- code octal ou hexadécimal du caractère : \173, \x7b (c’est un z)
- Non imprimables :

\n nouvelle ligne,
\r retour chariot,
\t tabulation horizontale,
\v tabulation verticale,
\b retour arrière, \f saut de page, \a signal d'alerte

# Le langage : **Les constantes**

Chaîne de caractères

```
char chaine[] = "Un texte un \
peu long";

char hello[6];
hello[0]='h';
hello[1]='e';
hello[2]='l';
hello[3]='l';
hello[4]='o';
hello[5]='\0';
```

# Le langage : **Les constantes**

Les énumérations

```
#include <stdio.h>
int main(){
enum naturelPair { ZERO, DEUX, QUATRE, SIX };
enum naturelPair z = ZERO;
enum naturelPair d = DEUX;
printf("%d \n", (int) z);
printf("%d \n", ZERO);
printf("%d \n", (int) d);
printf("%d \n", DEUX);
return 0;
}
```

# Le langage : les opérateurs

Affectation : =

```
#include <stdio.h>
//La fonction main
int main(){
int a=2,b = 2.1;
printf("%d %d \n",a,b);
printf("%d \n",a=44);
return 0;
}
```

# Le langage : les opérateurs

Les opérateurs arithmétiques : + addition, - soustraction, * multiplication, / division, % reste de la division (modulo)

Les opérateurs relationnels : > strictement supérieur, >= supérieur ou égal, < strictement inférieur, <= inférieur ou égal, == égal, != différent.

Les opérateurs logiques booléen : && et logique, || ou logique, ! négation logique.

Les opérateurs logiques bit à bit à ne pas confondre avec les booléens : & et, | ou inclusif, ^ ou exclusif, ~ complément à 1, << décalage à gauche, >> décalage à droite.

# Le langage : les opérateurs

- Les opérateurs d'affectation composée +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>= ils correspondent à l'opération suivi de l'affectation

```
#include <assert.h>
//La fonction main
int main(){
int a =2;
a +=10;
//Si a ne vaut pas 12, le programme s'arréte
//un message d'erreur est affiché
assert(a == 12);
return 0;
}
```

# Le langage : les opérateurs

Les opérateurs d'incrémentation ++ et de décrémentation --, sous forme suffixée la valeur retournée sera la nouvelle et sous la valeur préfixée sera l'ancienne.

```
//incrementation.c
#include <assert.h>
int main(){
int a =2;
assert(++a == 3);
assert(a == 3);
assert(a++ == 3);
assert(a == 4);
return 0;
}
```

# Le langage : les opérateurs

L'opérateur virgule, permet de réaliser plusieurs instructions en une seule expression évalué de la gauche vers la droite.

```
int a, b;
b = ((a = 10), (a + 4));
assert(b==14);
```

# Le langage : les opérateurs

L'opérateur conditionnel ternaire, sous la forme condition ? expression-1 : expression-2, l'évaluation de l'expression-1 est retournée si la condition est vraie et sinon l'expression-2 :

```
//ternaire.c
#include <assert.h>
int abs(int a) {
//Si a>0 alors a sinon -a
return a > 0 ? a : -a;
}
//La fonction main
int main(int argc, char *argv[]){
assert(abs(2)==2);
assert(abs(-2)==2);
assert(abs(0)==0);
return 0;
}
```

# Le langage : les opérateurs

L'opérateur de conversion de type, simplement sous la forme (type) objet :

```
int a = (int) 2.0; //2
float b = (float) 3/2; //1.5
```

# Le langage : les opérateurs

L'opérateur adresse & et l'opérateur d'indirection *. & retourne l'adresse mémoire et * la valeur référencée par

une adresse :

```
int a=10;
printf("valeur %d\n", a); //10
printf("adresse %p\n", &a); //0x16eee2eac
printf("valeur referencee par l'adresse %d\n",
 *(&a)); //10
```

# Le langage : **Les instructions de branchement conditionnel**

Le branchement conditionnel sous la forme :

```
if (expression)
instruction-1
[else
instruction-2]
```

```
/les accolade ne sont utiles que si il y a plus
//d'une instruction
int a=10;
if (a%2==0)
printf("a est pair\n");
else
printf("a est impair\n");
```

# Le langage : **Les instructions de branchement conditionnel**

Le branchement multiple :

```
switch (expression)
{case constante-1:
liste d'instructions 1
break;
case constante-2:
liste d'instructions 2
break;
...
case constante-n:
liste d'instructions n
break;
default:
liste d'instructions par def.
```

```
int value=10;
switch( value ) {
case 1:
printf( "One\n" );
break;
case 2:
printf( "Two\n" );
break;
case 3:
printf( "Three\n" );
break;
default:
printf( "Other value\n" ); //on passe ici
}
```

# Le langage : **Les boucles**

while (expression ) instruction

```
//Les n premiers nombres
int n = 4;
int i=0;
while (i < n) {
printf("%d\n",i);
i++;
}
//0 1 2 3
```

Rem : break permet de quitter la boucle, continue de passer à l'itération suivante mais les 2 sont à éviter?

# Le langage : **Les boucles**

do while (au moins un passage)

```
int n = 4;
int i=10;
do {
printf("%d\n",i);
i++;
} while (i < n);
//10
```

# Le langage : **Les boucles**

La boucle for est un cas particulier de la boucle while :

```
for (expr 1 ;expr 2 ;expr 3)
instruction
//est equivalent à
expr 1;
while (expr 2 )
{instruction
expr 3;
}
```

# Le langage : **Les boucles**

```
for (int i=0; i<5;i++)
printf("%d\n",i);
//0 1 2 3 4
```

# Le langage : les tableaux

Indicés, la première case est en position 0

Type nom[taille]

```
int tab[3] = { 10, 20, 30 };
//ou int tab[] = { 10, 20, 30 };

tab[0] = 10;
tab[1] = 20;
tab[3] = 30;
```

# Le langage : **Les entrées/sorties de base**

Printf pour écrire

printf("chaîne de contrôle ",expression-1, ..., expression-n);

Scanf pour lire

scanf qui est formatée comme printf mais prend en second paramètre un pointeur(une variable qui stocke une adresse)

```
float unFlottant;
printf("Saisir un flottant : ");
scanf("%f",&unFlottant);
printf("Le flottant %f  lu  \n",unFlottant);
```

# Le langage : **La ligne de commande**

Il est possible de passer des informations en ligne de commande à l'exécution d'un programme, c'est le rôle des paramètres argc et argv du main int main(int argc, char *argv[]).

argc donne le nombre d'éléments de la ligne de commande

argv[0] contient le nom de la commande, argv[i], pour i allant de 1 à argc-1, le ième argument.

# Le langage : **La ligne de commande**

```
int main(int argc, char *argv[]){
printf("Nombre de paramètres : %d\n",argc-1);
for (int i=1; i<argc; i++)
printf("Paramètre %d : %s \n",i,argv[i]);
}
```

Le programme précédent compilé en a.out et appelé comme suit ./a.out p1 p2 p3 contuira à l'affichage de

```
Nombre de paramètres : 3
Paramètre 1 : p1
Paramètre 2 : p2
Paramètre 3 : p3
```

# Le langage : **Structure d'un programme**

```
 [directives au préprocesseur]
[fonctions secondaires]
main() //ou int main(int argc, char *argv[])
{déclarations de variables internes
instructions
}
```

# Le langage : **Structure d'un programme**

```
#include <systemFilename.h> //inclure un fichier
 de déclaration standard
#include "userFilename.h" //inclure un fichier
 de déclaration personnel
```

# Le langage : **Structure d'un programme**

```
type ma_fonction ( arguments )
{déclarations de variables internes
instructions
}

int add(int a, int b) {
int s;
s = a+b;
return s;
}

void sayHello(){
printf("Hello");
}
```

# Exercice

Ecrire hello world

Ecrire hello world pour un nom saisi

# Exercice

Écrire un programme qui demande un âge et indique si la personne est majeure ou mineure, le programme demande ensuite un montant et applique les règles de gestion suivantes : les mineurs ont une remise de 5% sur le prix initial, si le prix initial est supérieur à 300 alors une remise de 5 Euros est appliquée.